CE/CS/SE 3354 · Software Engineering · Fall 2026

Project Milestone P2

Analysis and High-Level Design

[INTERNSHIP TRACKER(INTERNTRACK)]

Team	Team 10, section .003
Document version	1.0 (baseline for P2; grows from your P1 version 0.x)
Date	11/06/2026
Repository URL	https://github.com/nifemiolayiwola/InternTrack
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	Oluwanifemi Olayiwola

Team member	NetID	GitHub username
Oluwanifemi Olayiwola	ooo230009	nifemiolayiwola
Samuel Rodriguez	sxr230133	saro2301
Krish joshi	kxj230025	kjoshiDTX
Dayra Ponce	dcp230001	dayraponce
Ryan Tracy	rjt230004	ryantracy0
Harshith Reddy Tokala	hxt230033	tokalahr

0 Revision History, Contribution Record and Changes Since P1

Version	Date	Author	Summary of change
0.x	10/18/2026	Team 10	P1 baseline (copy your last P1 row)
1.0	11/08/2026	Team 10	P2 baseline: analysis and high-level design added

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Samuel Rodriguez	Sections 4.1- 4.3	https://github.com/nifemiolayiwola/InternTrack/blob/main/docs/p2/sam-p2.pdf	N/A - no PR review
Oluwanifemi Olayiwola	Sections 6.5-6.6, Sections 7	https://github.com/nifemiolayiwola/InternTrack/blob/main/docs/p2/Sections%206.5-6.6%20and%20Sections%207.pdf	N/A - no PR review
Ryan Tracy	Sections 6.1-6.4	https://github.com/nifemiolayiwola/InternTrack/blob/main/docs/p2/Sections%206.1%206.2%206.3%206.4.docx.pdf	N/A - no PR review
Krish Joshi	Sections 5.1-5.3	https://github.com/nifemiolayiwola/InternTrack/blob/main/docs/p2/Section%205%20(5.1-5.3)%20Analysis%20Models.pdf	N/A - no PR review
Harshith Reddy Tokala	Section 8	https://github.com/nifemiolayiwola/InternTrack/blob/main/docs/p2/Section%208.pdf	N/A - no PR review
Dayra Ponce	Sections 9.1-9.2	https://github.com/nifemiolayiwola/InternTrack/blob/main/docs/p2/sec%209.1-9.2.pdf	N/A - no PR review

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Section 2.1 context diagram updated to clarify system boundary and document storage	P1 context diagram feedback	Update context diagram and ADR-0.2	Agreed 11/08/2026
CH-02	Section 2.2 clarified assumed stakeholder needs and advisor access	P1 stakeholder evidence feedback	Stakeholder assumptions documented in section 11	Agreed 11/08/2026
CH-03	Section 3.4 keeps the use-case diagram, while detailed use cases move to section 4, section 3.5 refers to section 9.1	P2 requirements	Update Sections 4 and 9	Agreed 11/08/2026
CH-04	Added the missing quality requirements and project information to the Github repository	P1 repository feedback	Update repository documentation	Agreed 11/08/2026
CH-05	Team members worked on different sections for p2	The team divided the analysis and design tasks	Updated Section 10.1	Agreed 11/08/2026

1 to 3 Introduction, Overall Description and Requirements (carried forward)

1 Introduction

1.1 Purpose of this document and of the system

(unchanged since v0.x)

This document explains the project proposal and requirements for InternTrack, an Internship Application Management System. It describes the purpose of the project, the problem it helps to solve, the main features, user requirements, and possible challenges. It also explains how our team plans to develop the system

InternTrack is a web-based application system that is designed to help students keep track of their internship applications. Students often apply to internships through different company websites, which can sometimes make it difficult to remember application statuses, interview dates, and deadlines. InternTrack will help solve this problem by allowing students to save their applications, update their statuses, keep track of important dates, and view their overall progress in one place.

1.2 Scope

(unchanged since v0.x)

In scope — we will build this	Out of scope — we will not, and why
Allow students to create accounts and log in	University login systems because they require permission from the university
Create, view, edit and delete application	Automatically apply to internships because it requires access to company websites
Track status and status-change history(Applied, Interviewing, Offered, or Rejected)	Automatically check application statues because it would require connecting to other websites
Save interview dates and application deadlines	Automatically schedule interviews with employers because students will arrange interview themselves
Show reminders for upcoming deadlines	Send email or text reminders because they require outside services
Show application progress on a dashboard	Use AI to recommend internships because our main goal is to help students track their application
Search and filter internship applications	Automatically collect internship listings because it would require access to other job websites
Save resume and cover letter	Create or edit resumes and cover letters because students will upload documents they have already prepared

We excluded these features because some would require access to other websites or take more time to develop and test. Since this project must be completed within one semester, we want to focus on building the main features first. Our goal is to create a system that helps students to organize their internship applications, keep track of important dates, and view their progress all in one place. By focusing on these features, we can make sure the system is functional, easy to use, and completed within the project timeline.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
InternTrack	A web based application system designed to help students keep track of their internship applications, important dates, and overall progress in one place.
Status / Application Status	The current standing of a student's internship application. In this system, the status and status-change history are specifically tracked as Applied, Interviewing, Offered, or Rejected.
Dashboard	The central interface within the application where students can view their overall application progress.
Reminder	A notification displayed within InternTrack to help students remember upcoming interviews and deadlines

Term or abbreviation	Meaning as used in this project
QAS	Quality-attribute scenario
ADR	Architecture decision record

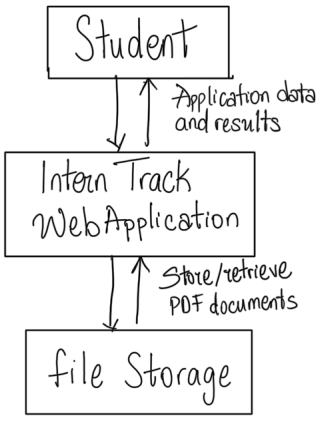
1.4 References

[R1] IEEE 29148: International standard for requirements engineering.
[R2] CS3354: P1 Proposal and Requirements instructions and rubrics
[R3] CS3354: P2 Analysis and High-level Design template and project description

2 Overall Description

2.1 Product perspective and context

InternTrack is a web-based application that allows students to track, store, and view their internship application progress. The student is the only direct human user in the current scope. A student enters application data, uploads PDF documents, and views saved applications, status history, reminders, and dashboard counts. University advisors and employers are indirect stakeholders, not users with accounts. The system owns the application records and permissions. The proposed architecture uses a separate file-storage service for PDF attachments.



2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Students	User who creates, and manages their applications. Primary user	Internship application history tracked in one place, with statuses (applied, interviewing, rejected, offered) for each application.	We assumed so from our own experiences, as the entire purpose of the app is to cater to students applying for internships.
University Advisors	Indirect stakeholder who may review application progress that a student chooses to share. Advisors do not have accounts in InternTrack.	Clear, visible application history data in order to make it easy for them to make proper judgement on what a student should do to enhance their job prospects.	We assumed so from our own experiences with career advisors and how difficult it can be to show application progress without an organized record.
Employers/Recruiters	Indirect stakeholder: not using the system, but affected by students tracking applications through it.	Students being honest and able to track their own application history, as users might apply to employer companies. Possibly keeping track if they have applied to their company, no double applications.	We assumed so from our experiences regarding the hiring and internship process, through our experiences speaking with recruiters at career fairs, and common recruiter grievances regarding the process.
Developers	Builders and later maintainers of both application and system.	Requirements clear and concise, agreed upon and streamlined development process, ability to add features without needing to majorly rework system.	We assumed so, as we are the ones that will be developing this application and we want to make the development process as smooth as possible.

No interviews or surveys were conducted. The stakeholder needs are based on our team's experiences and assumptions

2.3 User characteristics

(unchanged since v0.x)

Although students will be using the app for the same reasons (tracking internship applications and their progress), we must take into account that different kinds of students will be using this application.

1. Students newer to the application process: likely to not fully know how the application process works. They might not know how long until a follow up, the stages of the application process, or the fact that many apps do not get a response at all. It's imperative that our app streamline and display this information in a concise manner for easier use and to not feel overwhelming.

2. Experienced applicants: students who are more familiar with the application process. These students, once using the application, are likely to update their progress and add new entries at a much faster rate than the inexperienced. Adding and changing statuses of applications must be quick and take a few taps or clicks at most.

Generally, for every user, the application must have a clean user interface and user experience (as we will also have a dashboard). Text should be readable, color should fit the message the interface is trying to convey. Also assumed students do not want their information being shared.

2.4 Constraints

(unchanged since v0.x)

C1: P1 scope excludes single sign-on from the university and will instead use application-managed accounts.
Source: Section 1.2 scope decision

C2: The system shall not connect to employer portals or job boards to submit applications or automatically retrieve statuses.
Source: Section 1.2 exclusions

C3: Deadline reminders will be displayed inside InternTrack rather than sent by email or SMS.
Source: Section 1.2 exclusions

C4: The system will be accessible through a web browser, not a separately installed mobile application.
Source: Section 1.1 purpose of system

2.5 Assumptions and dependencies

(unchanged since v0.x)

#	Assumption or dependency	What breaks if it does not hold
A1	Assumption: students are willing to enter and update application details manually.	Statuses and reminders would become stale and the product offers little value without user entry.
A2	Assumption: students can supply their own application deadlines, interview dates, and status changes.	The calendar/reminder functions would not present accurate information.
A3	Dependency: users have a supported browser and internet access to reach the web application.	Users cannot access their records when offline or on unsupported devices.
A4	Assumption: no direct access to employer or university systems is needed for the agreed initial features.	Additional integrations would expand the scope and affect team feasibility.

2.6 Feasibility summary

(unchanged since v0.x)

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible	InternTrack's core features are common web application functions such as account management, status tracking, and dashboard summaries. The project will exclude automated job applications, third-party job board integrations, and AI recommendations, which reduces the technical complexity.
Operational	Feasible	InterTrack will provide one location for organizing application records and important dates. This web app will use a manual-entry approach which allows students to maintain their own information from multiple websites without requiring employers or universities to participate.
Schedule	Conditionally feasible	Feasibility depends on completing assigned work by the course deadlines and resolving any outstanding requirements before P2. Team can prioritize essential application functions before optional features.
Economic / resource	Feasible	The project can be developed using the team's computers, free development tools, and GitHub. No paid third-party APIs are required for the core features.

3 Specific Requirements

3.1 Functional requirements

(unchanged since v0.x)

ID	Requirement — one "shall" sentence	Priority	Source	Acceptance criterion
FR-01	The system shall allow a new user to create an account using an email address and a password of at least 8 characters	MUST	Students (2.2), Scope (1.2)	A tester registers with a valid email and an 8 characters password and can then log in. A 7 character password is rejected with an error message.
FR-02	The system shall reject a login attempt whose email and password do not match a registered account.	MUST	Students (2.2); Scope (1.2)	A tester enters a registered email with a wrong password. Access is denied and an error message appears. Correct credentials grant access.
FR-03	The system shall allow a logged-in student to create an application record containing company name, position title, and date applied.	MUST	Students (2.2); Scope (1.2)	A tester creates a record with all three fields. It appears in the application list with the values entered. Submitting with the company name blank is rejected.
FR-04	The system shall allow a student to edit any field of an existing application record.	MUST	Scope (1.2)	A tester changes the position title of a saved record. The new value shows after a page reload.
FR-05	The system shall delete an application record only after the student confirms the deletion in a confirmation prompt.	MUST	Scope (1.2)	Choosing "Cancel" leaves the record in the list. Choosing "Confirm" removes it, and it stays removed after a reload.
FR-06	The system shall restrict an application's status to exactly one of Applied, Interviewing, Offered, or Rejected.	MUST	Scope (1.2); Definitions (1.3)	The status control offers only these four values. A new record defaults to Applied.
FR-07	The system shall record the previous status, new status, and date of every status change on an application.	MUST	Scope (1.2)	A tester changes a record from Applied to Interviewing to Offered. The history view lists two entries in chronological order with correct dates.
FR-08	The system shall allow a student to store an application deadline and one or more interview dates on an application record.	MUST	Students (2.2); Scope (1.2)	A tester saves a deadline and two interview dates. All three show on the record after a reload.

FR-09	The system shall display a reminder for any application deadline or interview date in the next 7 days	SHOULD	Students (2.2); Scope (1.2); C3 (2.4)	A tester saves 2 applications with deadlines 3 and 10 days away respectively. Only the 3 day deadline appears in the reminders
FR-10	The system shall display a dashboard summary of the total applications and the count of applications in each status (Applied, Interviewing, Offered, Rejected)	MUST	Students (2.2); University Advisors (2.2); Scope (1.2); Definitions (1.3)	A tester creates 4 applications, one of each status. The dashboard shows the total count as 4 and 1 for each status
FR-11	The system shall allow a student to search applications by company name or position title and filter by application status.	SHOULD	Students (2.2); Scope (1.2)	Given 5 saved applications across different companies and statuses, by filtering using “Interviewing” or by searching by company name it displays only the matching records
FR-12	The system shall allow a student to upload and attach a PDF resume and cover letter of up to 10MB each to an application record	COULD	Students (2.2); Scope (1.2)	A tester uploads a 2 MB PDF resume. The file stays attached and downloadable. A 15MB file or a filetype that isn’t PDF is rejected.

3.2 Quality requirements

(unchanged since v0.x)

ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	95% of dashboard page loads shall complete within 2 seconds for an account holding up to 200 application records, measured over 100 loads on a standard broadband connection.	SHOULD	Students (2.2)	Seed a test account with 200 records. Time 100 loads with browser developer tools and compute the 95th percentile.
QR-02	100% of requests by one user to view, edit, or delete another user’s application record shall be denied.	MUST	Students (2.2)	Run a scripted set of at least 20 cross-account requests (direct URLs and API calls) from a second test account. Every request must return an access-denied response.

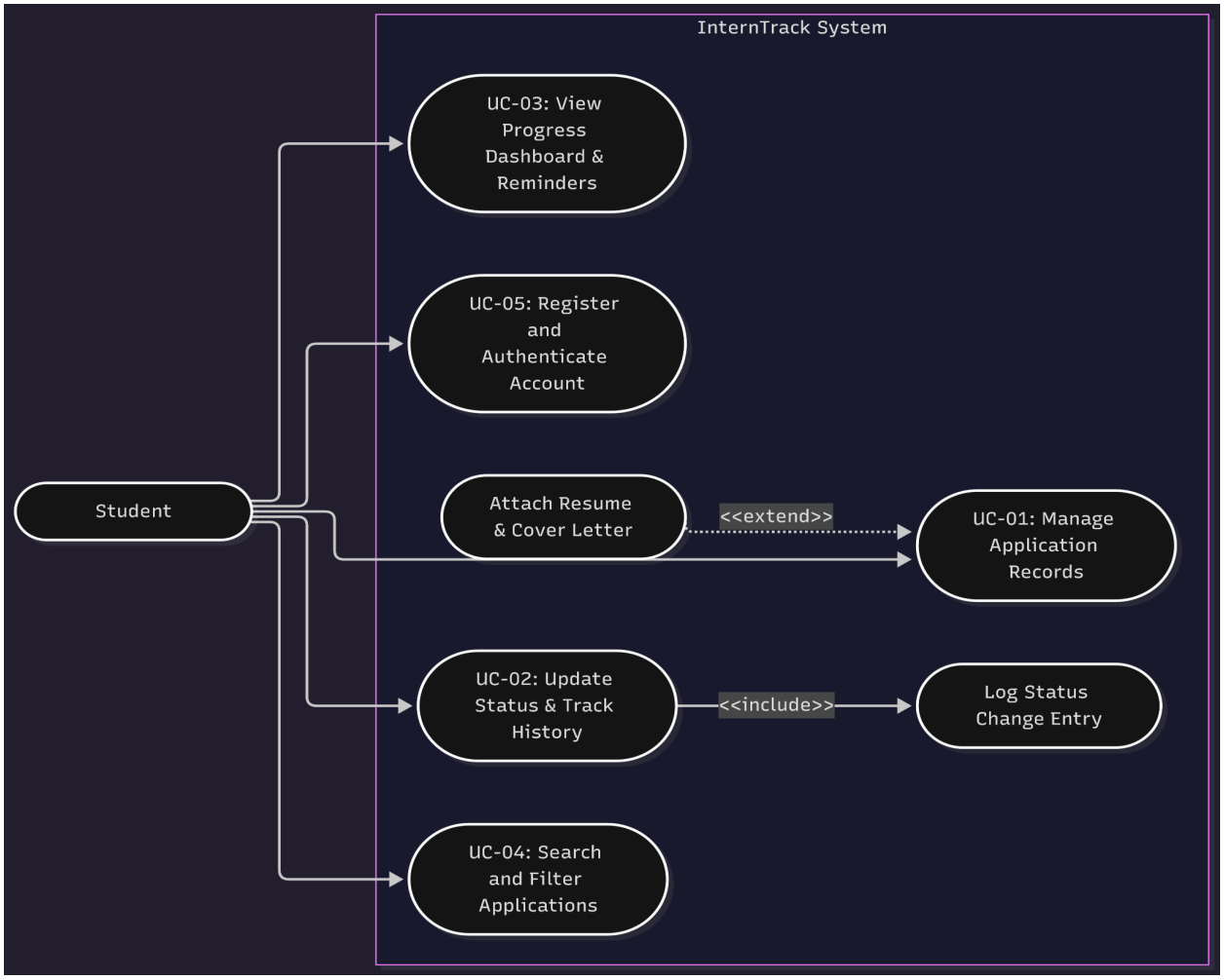
QR-03	No user password shall be stored in plaintext. Passwords shall be stored only as salted hashes produced by bcrypt or Argon2.	MUST	Students (2.2)	Register 5 test accounts, inspect the user table, and confirm that no stored value matches any entered password and every value has the hash algorithm's format.
QR-04	At least 4 of 5 first-time users shall create their first application record within 3 minutes without help.	SHOULD	User characteris tics (2.3)	Run a moderated usability session with 5 students who have not seen the system, and time each from login to saved record.

3.3 Constraint requirements(unchanged since v0.x)

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system shall authenticate users only through application-managed accounts and shall not use university single sign-on.	C1 (2.4); Scope (1.2)	University SSO needs approval from the university's IT office, which the team cannot get within the course timeline.
CON-02	The system shall not connect to any employer portal or job board to submit applications or retrieve their statuses.	C2 (2.4); Scope (1.2)	These sites need credentials or APIs the team has no access to, and automated access may violate their terms of service.
CON-03	The system shall display deadline and interview reminders only in the web interface and not send text or email notifications	C3 (2.4); Scope (1.2)	External notifying requires third party messaging services and paid APIs that are not within the scope's boundaries.
CON-04	The system shall be delivered only as a web application and not built as a mobile app	C4 (2.4); Purpose (1.1)	Building and maintaining a mobile app alongside a web app exceeds the team's schedule constraints for the semester

3.4 Initial use cases

(unchanged since v0.x)



Use case	Name	Primary Actor
UC-01	Manage Internship Application Record	Student
UC-02	Update Application Status and Track History	Student
UC-03	View Progress Dashboard & Reminders	Student
UC-04	Search and Filter Applications	Student
UC-05	Register and Authenticate Account	Student

The detailed descriptions of the selected use cases are provided in Sections 4.2.

3.5 Requirements to use case traceability

The requirements-to-use-case traceability matrix has been moved to Section 9.1. The update matrix connects the requirements to use cases, analysis classes, architecture components, quality scenarios, design decisions, and evidence.

4 Use-Case Analysis

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-01	Manage Internship Application Record	This is what our application is meant to do at the most basic level.	Inputting required fields, detecting duplicate applications, linking apps to logged in users.
UC-02	Update Application Status and Track History	Application statuses change, and the user must be able to reflect that.	Check if status is one of allowed statuses, not allowing same status changes (applied cannot change to applied)
UC-05	Register and Authenticate Account	Application information must be unique to each user logged in.	Check if information isn't taken, hash password, keep user logged into session

4.2 Expanded use cases

UC-01: Manage Internship Application Record

Primary actor	Student
Supporting actors and external systems	None (as we have no external systems)
TUCBW (this use case begins when)	The use case begins when a student chooses to add a new application or edit/delete an existing one.

TUCEW (this use case ends when)	This use case ends when the student sees requested changes reflected in the application.
Preconditions	The student must have an account and must be logged in. For editing and deleting, the user must have already created an application entry.
Postconditions	If successful, a record is created or saved and reflected in the application, and if “delete” then the application is deleted. If not successful, records are unchanged.
Traces to requirements	FR-03, FR-04, FR-05, FR-06, FR-08, FR-12, QR-02, QR-04, no CON

Actor input / action	System response
	(0) system displays applications list with “add application” option
(1) student selects “Add Application”	(2) system asks for: company name, position title, date applied, deadline, dates for interview, resume and cover letter upload.
(3) Student enters: company name, position title, date applied. Also enters: deadline, interview dates, resume, cover letter, then submits application.	(4) System checks files are PDF less than or equal to 10MB space, and if required fields are filled.
	(5) System reflects changes in application
	(6) System displays application new list
(7) Student sees the new application in their list.	

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	1	student wants to edit application	student chooses to edit an application’s information and system displays details to change	Rejoins at 3, changes reflected in 7
A2	1	student wants to delete application	The student chooses to delete an application after choosing it. The student must then confirm if they want to delete. If so, delete, if not, do not.	Ends

UC-02: Update Application Status and Track History

Primary actor	Student
Supporting actors and external systems	None

TUCBW (this use case begins when)	The use case begins when the student chooses an existing application.
TUCEW (this use case ends when)	The use case ends when the system displays application edit, or if the student cancels, nothing changes.
Preconditions	The student must be logged in and have at least one application logged.
Postconditions	App shows new changes/edits in the record if successful, and no changes if not successful.
Traces to requirements	FR-04, FR-06, FR-07, FR-08, QR-02

Actor input / action	System response
	(0) system displays all student applications
(1) student chooses application	(2) system displays application info (status: applied, rejected, offered, interviewing)
(3) student chooses new status or new info to fill in and saves it	(4) system checks if new info differs from old info
	(5) system updates current status and adds history entry with new status, previous, and date.
	(6) system displays new status and updated history
(7) student now sees both.	

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	4	Same status selected	System tells student that nothing has changed and adds no history entry.	ends
A2	3	Student chooses Interviewing	The system asks the student for interview dates, then dates are saved if inputted, with a reminder once within 7 days of interview date.	Rejoins at 5

UC-05: Register and Authenticate Account

Primary actor	Student
Supporting actors and external systems	None (as we manage accounts, not UTD SSO)
TUCBW (this use case begins when)	The student opens InternTrack and chooses to make an account/log in.

TUCEW (this use case ends when)	The student sees the updated status and history, or when the student cancels without making changes
Preconditions	The student must not be logged in
Postconditions	The account is then created (if registering) or logged into if the account already exists. If it fails, no session is started or no account is created.
Traces to requirements	FR-01, FR-02, QR-02, QR-03, CON-01

Actor input / action	System response
	(0) System displays login message with an option to register
(1) Student chooses to create account	(2) System asks for email and password fields to fill out
(3) Student fills out both fields	(4) Check if the email is unique, and that the password is at least 8 characters.
	(5) if so, the system creates an account.
	(6) system tells student account created and to log in
(7) The student enters email, password and logs in.	(8) Check if email and password combinations match an account.
	(9) system now displays the student's applications
(10) student now sees their applications	

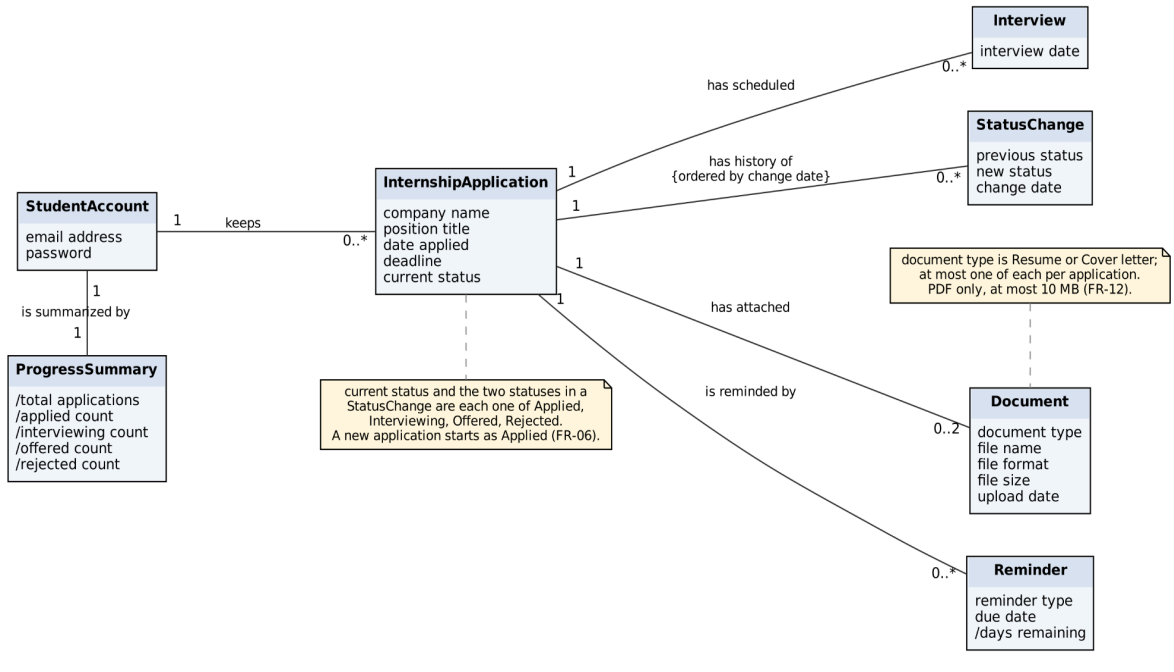
Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	1	Student already has account	The student skips account creation and goes straight to login	Rejoins at 7
A2	8	Email and password combination don't link to an account	System does not grant access to the student and asks them to try again	Rejoins at 7
A3	4	Email is already registered	System tells the student the email is already in use and asks for another email	Rejoins at 3
A4	4	Password does not meet requirements	System displays an error message and asks the student to enter a valid password	Rejoins at 3

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-01/UC-02	Is the actor a role?	Put in “student/registered user” but registered user is not a role.	Changed to just “student”
All three	One turn per row?	Final system row had many functions at once like checking, displaying, recording	Multiple rows for each
UC-02	No background processing?	Included reminder updates student won’t see	Removed as they belong to UC-03
UC-05	Are error cases included?	Missing steps for an email already in use or invalid password	Added A3 and A4 to show what happens

5 Analysis Models

5.1 Domain model



Insert diagram here, then replace this frame with the image.

Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Student account	A student's registered InternTrack account, the owner of every record below it	Email address, password	keeps 0..* InternshipApplication; is summarized by 1 ProgressSummary
InternshipApplication	One internship that the student applied ti	company name, position title, date applied, deadline, current status	kept by 1 StudentAccount; has scheduled 0..* Interview; has history of 0..* StatusChange; has attached 0..2 Document; is reminded by 0..* Reminder
interview	One interview scheduled for an application	Interview date	Scheduled for 1 InternshipApplication
StatusChange	One entry in an application’s status history	Previous status, new status, change date	Belong to 1 InternshipApplication, ordered by change date
Document	A resume or cover letter the student attached to an application	Document type, file name, file format, file size, upload date	Attached to 1 InternshipApplication
Reminder	An in app notice for a deadline or interview date 0 to 7 days away	Reminder type, due date, /days remaining	About 1 InternshipApplication
ProgressSummary	The student’s dashboard counts, worked out from their applications	/total applications, /applied count, /interviewing count, /offered count, /rejected count	Summarizes 1 StudentAccount

5.2 Object interaction models

UC-01 and UC-02 are carried through. Together they realize seven of the twelve functional requirements (FR-03 to FR-08 and FR-12). UC-02 holds the riskiest step: keeping an application's status and its history in step. Each diagram adds one interaction object to the classes of 5.1:

5.2.1 UC-01 Manage Internship Application Record

(a) Non-trivial steps. This analysis follows UC-01 in 4.2: the add flow (0) to (6) and alternate flow A2 (delete).

- (0) needs the student's own records, so ApplicationManager asks StudentAccount for them. Only that student's records appear (QR-02).

- (4) needs the file check (FR-12) and the required-field check (FR-03).
- (5) needs a new InternshipApplication with status Applied, its Interview and Document entries, and the link to StudentAccount (FR-03, FR-06, FR-08, FR-12), then the updated list.
- A2 needs the student's confirmation before StudentAccount gives up the record (FR-05).
- (1) and (2) are trivial: the student picks Add Application and sees an empty entry with no stored data. (3) is the student's input and appears below only as the message that starts (4). (6) is the student reading the list.
- A1 (edit) reuses rows 3 to 5.5: the record's details are shown, the same checks run at (4), and at (5) the existing InternshipApplication is updated instead of created. It is not drawn again.
- 4.1 lists duplicate detection as non-trivial, but 4.2 has no step for it and no requirement asks for it, so it is not modeled here (see 5.3, K5).

(b) Scenario table. Rows starting with A2 belong to alternate flow A2 in 4.2, where the student deletes an application.

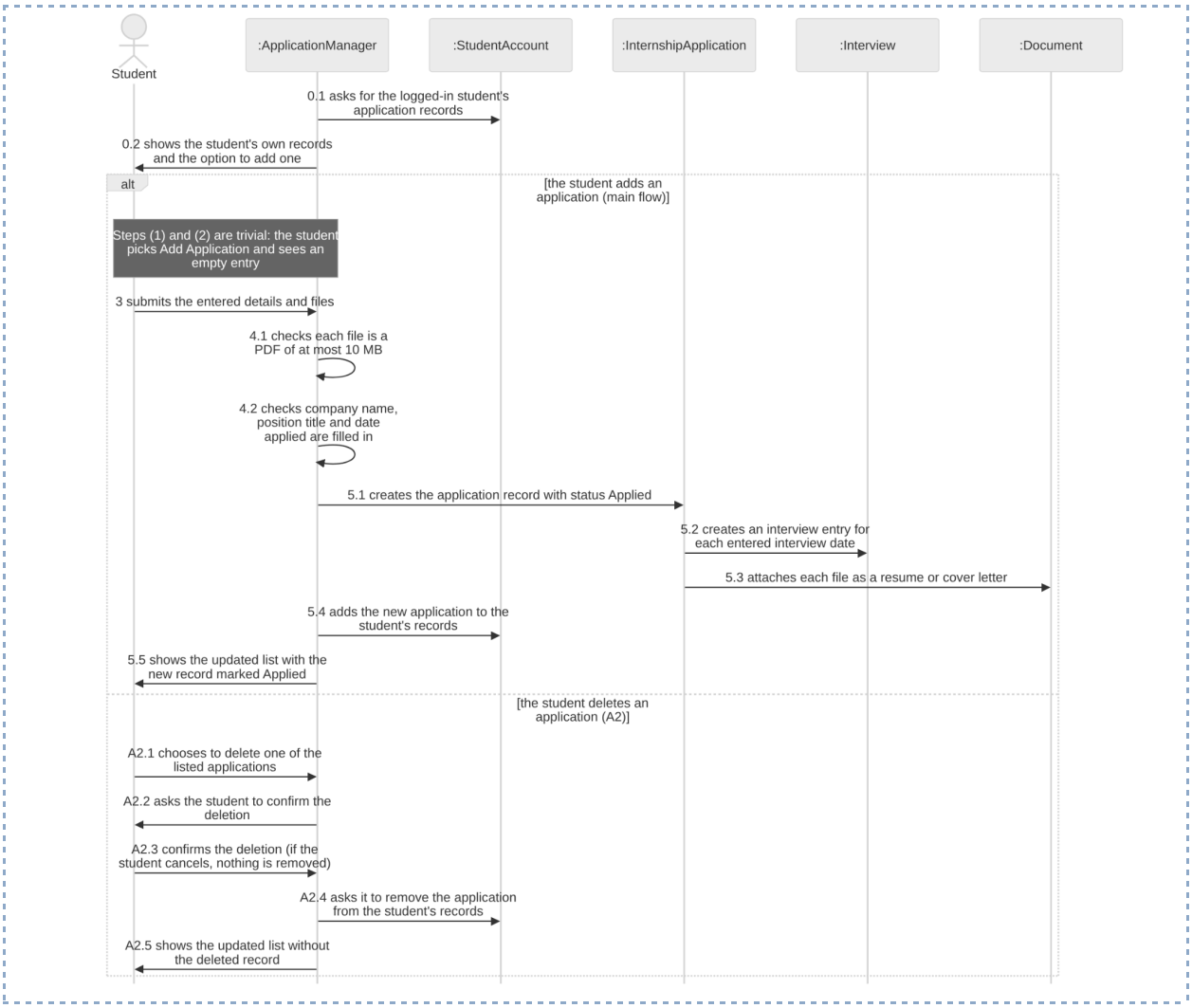
Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
0.1	ApplicationManager	asks for the logged-in student's application records	—	StudentAccount
0.2	ApplicationManager	shows the student's own records and the option to add one	company name, position title and current status of each record	Student (actor)
3	Student (actor)	submits the entered details and files	company name, position title, date applied, deadline, interview dates, resume file, cover letter file	ApplicationManager
4.1	ApplicationManager	checks each file is a PDF of at most 10 MB	file format, file size	ApplicationManager (itself)
4.2	ApplicationManager	checks company name, position title and date applied are filled in	entered details	ApplicationManager (itself)

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
5.1	ApplicationManager	creates the application record with status Applied	company name, position title, date applied, deadline	InternshipApplication
5.2	InternshipApplication	creates an interview entry for each entered interview date	interview date	Interview
5.3	InternshipApplication	attaches each file as a resume or cover letter	document type, file name, file format, file size, upload date	Document
5.4	ApplicationManager	adds the new application to the student's records	the new InternshipApplication	StudentAccount
5.5	ApplicationManager	shows the updated list with the new record marked Applied	updated list of records	Student (actor)
A2.1	Student (actor)	chooses to delete one of the listed applications	the chosen application	ApplicationManager
A2.2	ApplicationManager	asks the student to confirm the deletion	company name and position title of the chosen application	Student (actor)
A2.3	Student (actor)	confirms the deletion (if the student cancels, nothing is removed)	—	ApplicationManager
A2.4	ApplicationManager	asks it to remove the application from the student's records	the chosen InternshipApplication	StudentAccount

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
A2.5	ApplicationManager	shows the updated list without the deleted record	updated list of records	Student (actor)
0.1	ApplicationManager	asks for the logged-in student's application records	—	StudentAccount
0.2	ApplicationManager	shows the student's own records and the option to add one	company name, position title and current status of each record	Student (actor)

(c) Analysis sequence diagram.

Analysis sequence diagram. ApplicationManager is in control. The 15 messages are the 15 rows above, with the same numbers and wording.



5.2.2 UC-02 Update Application Status and Track History

(a) Non-trivial steps. This analysis follows UC-02 in 4.2: turns (0) to (7), with A1 (same status chosen, at step 4) and A2 (Interviewing chosen, at step 3).

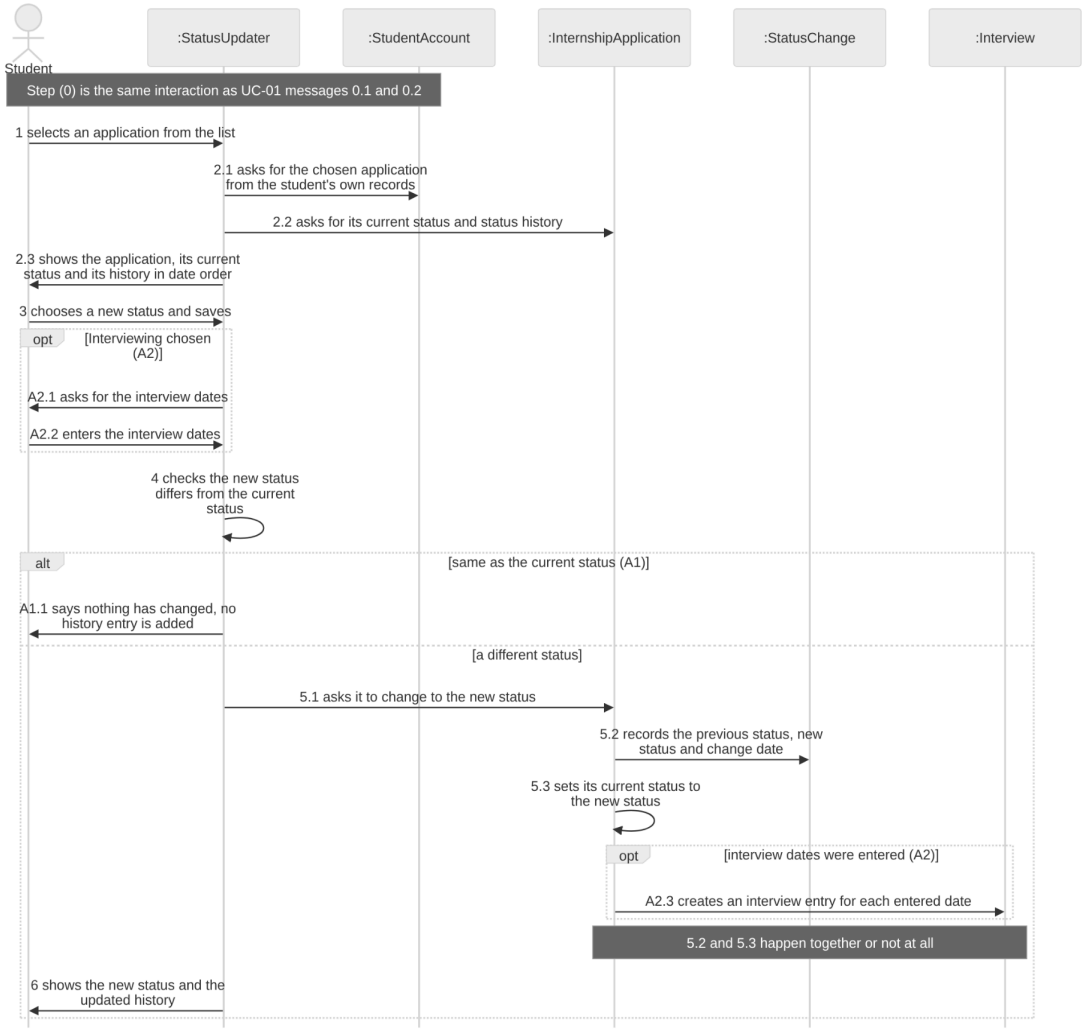
- (0) is the same interaction as UC-01 rows 0.1 and 0.2 and is not repeated.
- (2) needs the chosen application from the student's own records (QR-02) and its current status and history (FR-07).
- (4) needs the same-status check (FR-06).
- (5) needs a new StatusChange and the new current status, kept together so they never disagree (FR-06, FR-07). In A2 it also needs an Interview entry for each entered date (FR-08).
- (1) and (3) are the student's input and appear below only as the messages that start (2) and (4). (6) shows what (5) produced, and (7) is the student reading it.
- New details other than status at (3) follow the UC-01 edit flow (A1) and are not repeated. Reminders for the new interview dates belong to UC-03.

(b) Scenario table. Rows starting with A1 or A2 belong to those alternate flows in 4.2.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	Student (actor)	selects an application from the list	the chosen application	StatusUpdater
2.1	StatusUpdater	asks for the chosen application from the student's own records	the chosen application	StudentAccount
2.2	StatusUpdater	asks for its current status and status history	—	InternshipApplication
2.3	StatusUpdater	shows the application, its current status and its history in date order	company name, position title, current status, StatusChange entries	Student (actor)
3	Student (actor)	chooses a new status and saves	new status	StatusUpdater
A2.1	StatusUpdater	asks for the interview dates	—	Student (actor)
A2.2	Student (actor)	enters the interview dates	interview dates	StatusUpdater
4	StatusUpdater	checks the new status differs from the current status	current status, new status	StatusUpdater (itself)
A1.1	StatusUpdater	says nothing has changed, no history entry is added	current status	Student (actor)

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
5.1	StatusUpdater	asks it to change to the new status	new status, change date, interview dates	InternshipApplication
5.2	InternshipApplication	records the previous status, new status and change date	previous status, new status, change date	StatusChange
5.3	InternshipApplication	sets its current status to the new status	new status	InternshipApplication (itself)
A2.3	InternshipApplication	creates an interview entry for each entered date	interview date	Interview
6	StatusUpdater	shows the new status and the updated history	new status, StatusChange entries	Student (actor)

c) Analysis sequence diagram. StatusUpdater is in control. Messages 4.3, 4.4 and A2 are kept together or not at all, so A3 leaves the record exactly as it was. The 17 messages are the 17 rows above.



5.3 Consistency cross-check

Three checks failed on the first pass and were fixed (K1, K2, K4). The other three pass, and K5 found a step with no requirement behind it.

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	Failed, then passed. An earlier draft of UC-01 step (2) asked only for a resume, while FR-12 also allows a cover letter. Step (2) now asks for both, and Document.document type covers both. All other items match: company name, position title, date applied, deadline (InternshipApplication); new status (InternshipApplication, StatusChange); interview dates (Interview); email address and password (StudentAccount, UC-05).	4.2 UC-01 step (2); 5.1 Document
K2 Every object on a sequence diagram is a domain class or a declared interaction object	Failed, then passed. The first draft named the domain class Student, the same as the actor, so the diagrams could not tell the person from the account record. It is now StudentAccount. ApplicationManager and StatusUpdater are declared in 5.2.	5.1, 5.2.1, 5.2.2
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass. The UC-01 diagram has 15 messages and its table 15 rows; UC-02 has 14 and 14. Numbers and wording match, alternate-flow rows included. An earlier draft worded three UC-01 labels differently in the table and the diagram; they now match.	5.2.1 (b)
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	Failed, then passed. After 4.2 was revised (submitting moved into step (3), both checks into (4), saving into (5)), 5.2 still used the old step numbers. Both tables and diagrams were renumbered. Now UC-01: (0) rows 0.1–0.2, (4) rows 4.1–4.2, (5) rows 5.1–5.5, A2 rows A2.1–A2.5. UC-02: (0) reuses UC-01 rows 0.1–0.2, (2) rows 2.1–2.3, (4) row 4, (5) rows 5.1–5.3.	5.2.1, 5.2.2

Check	Result (pass, or what failed)	Fixed in
K5 Every analyzed use case traces to at least one functional requirement	Pass. UC-01 traces to FR-03, FR-04, FR-05, FR-06, FR-08, FR-12, and UC-02 to FR-04, FR-06, FR-07, FR-08, matching their trace rows in 4.2. Finding: 4.1 lists duplicate detection as non-trivial for UC-01, but no requirement asks for it and 4.2 has no step for it, so 5.2.1 does not model it.	Open question Q4 in section 11
K6 The last messages of each diagram make the use case postcondition true	Pass. UC-01: 5.1 creates the record, 5.4 links it to the student and 5.5 shows it. In A2, A2.4 removes it and A2.5 shows the list without it; a cancel at A2.3 leaves records unchanged. UC-02: 5.2 and 5.3 change the history and status together, and 6 shows both; A1 changes nothing.	—

6 High-Level Design (Architecture)

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	A logged in student with 200 application records opens the dashboard	Dashboard and Data Access Component	The system shows the total count, the count in each of the 4 statuses, and reminders for the next 7 days	95% of 100 dashboard loads complete within 2 seconds	QR–01
QAS-02	Security (authorization)	A logged in student sends a request to view, edit, delete, or download another student's application record	Authentication Component (AccessCheck) and every component that returns or changes records	The system refuses the request and nothing is changed or given out	100% of at least 20 cross-account requests are denied	QR-02

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-03	Security (confidentiality)	An attacker obtains a full copy of the stored account data	Authentication component and the stored account data	No password can be read from the stored data	0 of the stored values match any entered password.	QR-03
QAS-04	Usability	A first time student logs in and wants to record an internship	User interface and application management	The student saves the record and sees it in the list with status applied without help.	At least 4 of 5 first time users save their first record within 3 minutes of login	QR-04

Constraints: CON-1 means InterTrack owns registration, login and password storage, so an authentication component is required as QAS-03 applies to our data. CON-02 leaves the file storage service as the only external system. CON-03 means reminders are computed inside the system and no messaging service is needed.

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Layered: Browser layer, server layer, data access layer	QAS-02: Every request enters through one application layer, where AccessCheck runs before data is read. QAS-04: One server and one database keep the dashboard to a single path	Each request passes through every layer, which adds some pass through code and processing time. The single application server is a single failure, and changes redeploy the entire server.	Chosen
Microservices: one service per component	Independent deployment and scaling of each service	6+ services to build, deploy with a 5 person team in one semester.	Rejected
Two tiered Client server	Fastest to build, fewest components, low latency for QAS-01	The browser is untrusted, so QAS-02 would entirely require the database rules written correctly. Many rules would run in the client and could be bypassed.	Rejected

We chose the layered client server because our two MUST quality scenario are both security related. QAS-02 requires 100% of all cross account requests to be denied, and the layered style gives a single place that every request must pass before it gets to access data. That makes the access check impossible to skip from the browser, which is the primary weakness of the two tiered client server option.

6.3 Component view

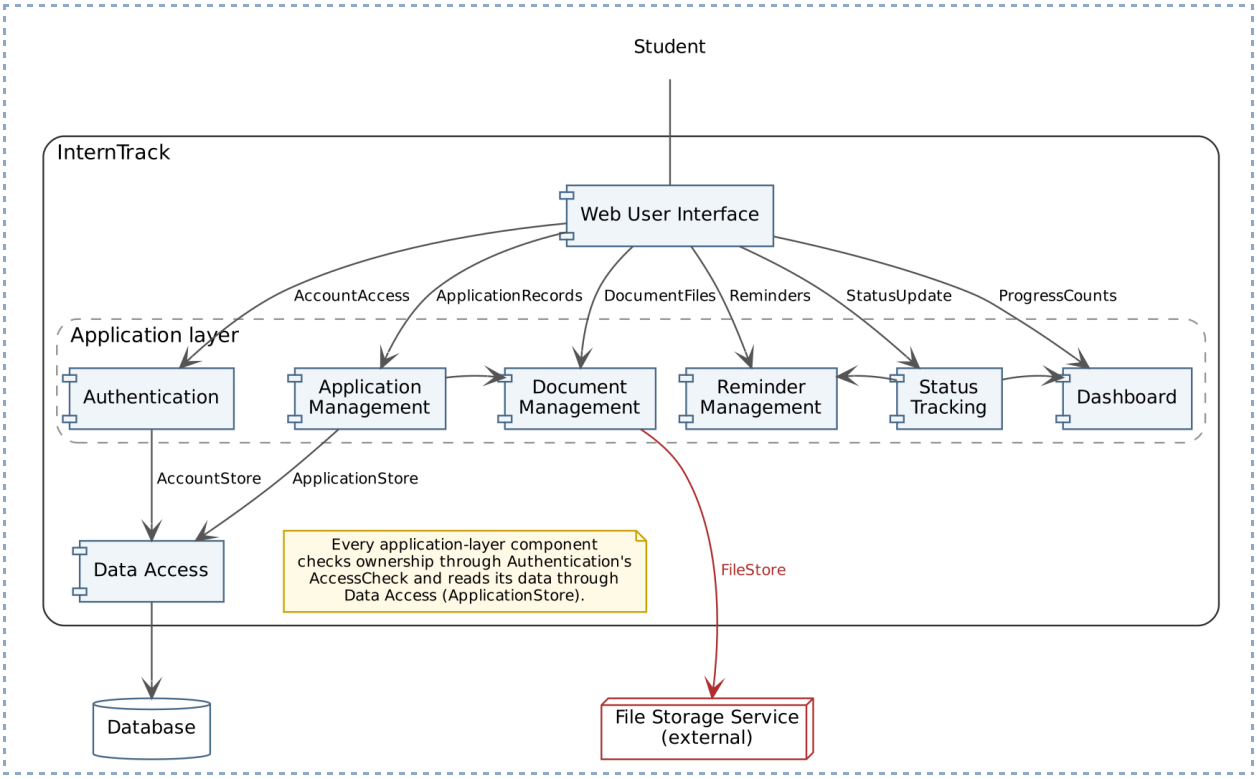


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Web User Interface	Shows register and login pages, application list, add and edit form, status history, reminders and dashboard	Pages to the student	All 6 interfaces	UC-01-UC-05, QAS-04, CON-04, ADR-01
Authentication	Registers accounts, hashes and verifies passwords, decides whether a requested record belongs to the student	AccountAccess, AccessCheck	AccountStore	UC-05, QAS-02, QAS-03, CON-01
Application Management	Creates, shows, edits, and deletes application records and interview dates	ApplicationRecords	AccessCheck, ApplicationStore, DocumentFiles	UC-01, UC-04, QAS-02, QAS-04

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Status Tracking	Changes an application's status, denies changing to the same status, and records previous statuses.	StatusUpdate	AccessCheck, ApplicationStore, Reminders, ProgressCounts	UC-02, QAS-02
Document Management	Accepts PDFs of at most 10MB and stores and retrieves them through the file storage	DocumentFiles	AccessCheck, ApplicationStore, FileStore	UC-01, CON–03
Reminder Management	List student's deadlines and interviews in the next 7 days	Reminders	AccessCheck, ApplicationStore	UC-03, ADR-03, CON-03
Dashboard	Counts the student's applications	ProgressCounts	AccessCheck, ApplicationStore	UC-03, QAS-01
Data Access	Only component that reads and writes to the database	AccountStore, ApplicationStore	Database	All UCs, QAS-01, QAS-02, ADR-01
File Storage Service	Holds the uploaded PDF files	FileStore	-	FR-12, ADR-02

6.4 Deployment view

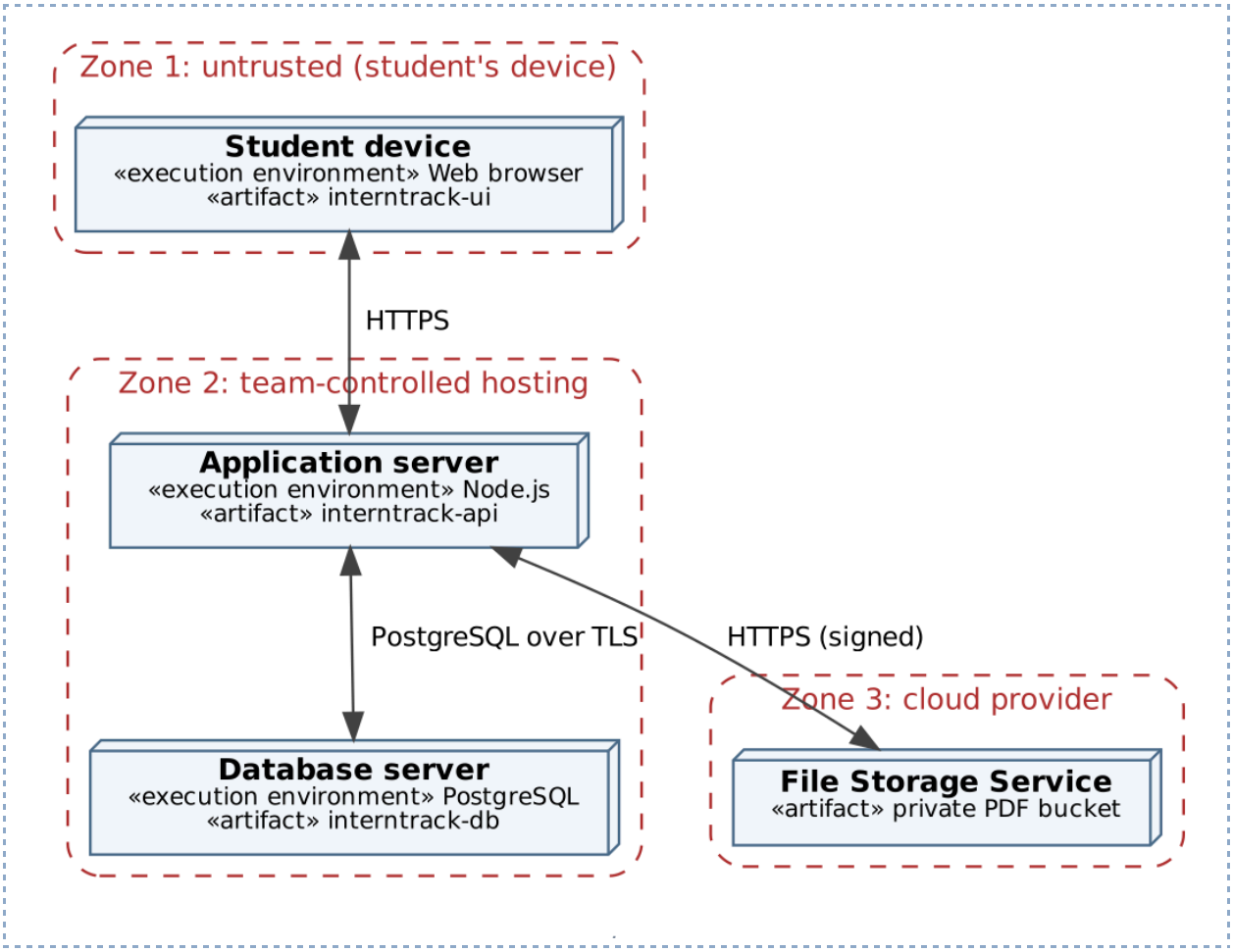


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
Student device (phone/computer)	Artifact interntrack-ui	Application server	Zone 1, untrusted. Everything from here is validated on server
Application server	Artifact interntrack-api, which is authentication application management, status tracking, document management, reminder management, dashboard, data access	Browser, database server, file storage service	Zone 2, team controlled

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
Database server	Artifact interntrack-db: accounts, applications, interviews, status changes, document details	Application server	Zone 2, team controlled
File Storage Service	PDF files	Application server	Zone 3

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
StudentAccount	Authentication Component	Manages student accounts, login information and access to the system
InternshipApplication	Application Management Component	Handles creating, viewing, editing, and deleting internship applications
Status Change	Status Tracking Component	Updates application statuses and stores the history of status change
Interview	Application Management Component	Stores interview dates and other important details related to an application
Document	Document Management Component	Handles PDF uploads and storage
Reminder	Reminder Management Component	Handles upcoming deadlines and interviews
ProgressSummary	Dashboard	Displays application progress and counts

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Layered architecture in Section 6.2 and component diagram in 6.3	Layered architecture(ADR-01), web user interface, data access
Information hiding	Authentication and Document Management Components	Account and file storage details are managed within their components
Low coupling	Component interfaces in section 6.3	Application Management uses the AccessCheck and ApplicationStore interfaces instead of directly handling authentication and database access

Principle	Where it shows in your design	Evidence (component, interface, ADR)
High cohesion	Application Management, Status tracking and Reminder Components	Each component focuses on related tasks
Design for change	Layered architecture and separate document storage	Layered architecture(ADR-01) and separate document storage(ADR-02) allow components to be updated independently.
Keep it simple (KISS)	Layered architecture selected in Section 6.2	Avoids unnecessary microservices and complex integrations

7 Design Rationale

7.1 Architecture decision records

ADR-01 [Layered Architecture]

Status	Accepted
Date and owner	11/07/2026 , Oluwanifemi Olayiwola
Context (drivers)	InternTrack needs to manage student accounts, internship applications, reminders, and uploaded documents. The system should be organized, secure and easy to maintain as new features are added
Options considered	Layered architecture, two-tier client server and microservices
Decision	We plan to use a layered architecture because it keeps the system organized and makes it easier to develop, test, and update different parts of InternTrack
Consequences	The system is easier to build, test and update. The requests may take longer because they pass through different layers. If the application server fails, the system may stop working
Evidence	Architecture comparison in 6.2 and diagrams in 6.3- 6.4
Related	QAS-01, QAS-02, QAS-03

ADR-02 [Separate Document Storage]

Status	Accepted
Date and owner	11/07/2026 , Oluwanifemi Olayiwola
Context (drivers)	FR-12, QR-02, document storage requirements

Options considered	• Separate File Storage: stores the actual documents separately and saves their information in the database • Database Storage: stores the documents directly in the database with the other application information
Decision	We plan to use separate file storage because it keeps documents organized and makes them easier to manage.
Consequences	This makes the system easier to maintain and update document storage. However, we need to make sure the documents stay connected to the correct student and application. The application can become a
Evidence	Context diagram in Section 2.1 and document storage design in sections 6.3-6.4
Related	FR-12, QR-02, QAS-02, CH-01

ADR-03 [In-App reminders]

Status	Accepted
Date and owner	11/07/2026 , Oluwanifemi Olayiwola
Context (drivers)	FR-09 requires reminders for upcoming interviews and deadlines within seven days. CON-03 excludes email and text notifications.
Options considered	• In-App Reminders: Shows reminders inside InternTrack when students use the website • External Notifications: Sends reminders through email or text messages.
Decision	We plan to use in-app reminders because they are simpler to build and do not need outside services.
Consequences	This keeps the system simple and makes it easy to maintain. But students will need to open the InternTrack to see their reminders.
Evidence	FR-09 and CON-03 in section 3, UC-03 in section 3.4 and reminder management in section 6.3
Related	FR-09, CON-03, UC-03

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Maintainability and security	Use layered architecture to separate the interface, application rules, domain logic, and data storage	Requests must pass through multiple layers, which may increase processing time	ADR-01

Quality favored	Architectural move	Cost accepted	Recorded in
Maintainability and flexibility	Keep uploaded documents in separate file storage and hide storage details behind the Document Management component.	Additional file management and permission checks	ADR-02
Simplicity	Use dashboard reminders instead of email or text notifications	Students must open the website in order to see reminders.	ADR-03

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01 - Dashboard Performance Test
Risk or decision it addresses	QAS-01
Question (answerable yes or no)	Can the chosen Node.js and PostgreSQL backend aggregate and return dashboard counts for an account with 200 application records in under 2 seconds?
Set-up	A Node.js Express server connected to a local PostgreSQL database, seeded with 1 test user and 200 dummy application records spanning all four statuses. Code and database schema are located in /spikes/S-01/
How it was measured	A script sent 100 sequential HTTP GET requests to the dashboard endpoint. The response times were recorded using a performance profiling tool
Result (numbers)	The 95th percentile response time was 42ms, and the maximum was 87ms
What it changed or confirmed	Confirmed that the layered architecture and chosen database can easily meet the 2-second threshold required by QAS-01 without needing additional caching mechanisms.
Where to see it	Folder /spikes/S-01/ (https://github.com/nifemiolayiwola/InternTrack/tree/main/spikes/S-01)

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	UC-05	StudentAccount	Authentication	QAS-03	Section 4

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-02	UC-05	StudentAccount	Authentication	QAS-02, 03	Section 4
FR-03	UC-01	InternshipApplication	Application Management, Data Access	QAS-04	Section 5.2.1
FR-04	UC-01	InternshipApplication	Application Management	QAS-02	Section 5.2.1
FR-05	UC-01	InternshipApplication	Application Management	QAS-02	Section 5.2.1
FR-06	UC-02	InternshipApplication, StatusChange	Status Tracking		Section 5.2.2
FR-07	UC-02	StatusChange	Status Tracking		Section 5.2.2
FR-08	UC-01	InternshipApplication, Interview	Application Management		Section 5.2.1
FR-09	UC-03	Reminder	Reminder Management	ADR-03	Section 4
FR-10	UC-03	ProgressSummary	Dashboard	QAS-01	Section 4
FR-11	UC-04	InternshipApplication	Application Management		Section 4
FR-12	UC-01	Document	Document Management, File Storage Service	ADR-02	Section 5.2.1
QR-01	UC-03	ProgressSummary	Dashboard	QAS-01	Section 6.1
QR-02	UC-1,2,3	StudentAccount	Authentication	QAS-02	Section 6.1
QR-03	UC-05	StudentAccount	Authentication	QAS-03	Section 6.1
QR-04	UC-01	InternshipApplication	Web UI, Application Management	QAS-04	Section 6.1
CON-01	UC-05	StudentAccount	Authentication	ADR-01	Section 2.4
CON-02	UC-1,2	InternshipApplication	Application Management	ADR-01	Section 2.4
CON-03	UC-03	Reminder	Reminder Management	ADR-03	Section 2.4
CON-04	All	—	Web UI	ADR-01	Section 2.4

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	None identified. All 20 requirements are assigned to at least one component in Section 9.1.	Reviewed the traceability matrix to confirm that each functional, quality, and constraint requirement has an assigned component.
Components with no requirement, QAS or ADR	None. All nine components are accounted for in Section 6.3.	Cross-checked the components against their associated requirements, use cases, QAS, and ADR references.
Use cases not analyzed (and why that is acceptable)	UC-03, UC-04	These use cases were included in the design but were not expanded in Section 4.2. UC-01 02, and 05 were prioritized as they covered core application management, status tracking, and authentication.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
Oluwanifemi	Introduction, scope and review documents	The team will discuss which features to include in our scope and vote if we all cannot agree on the same one
Harshith	Work on definition, references, and help review requirements	We will review the project goals and agree on what information is needed
Samuel	Review stakeholders, user needs and system constraints	We will discuss what students(users) need and choose the best option
Dayra	Feasibility, assumptions and project requirements	We will compare different ideas and choose the most realistic and feasible one
Ryan	Functional requirements and help update use cases	We will review the requirements and choose what works best for the system
Krish	Use cases, traceability and project documentation	We will discuss different ideas and agree on what best matches the requirements

For P2, the team members worked on different sections of the project. Samuel worked on Section 4, Krish worked on Section 5, Ryan worked on Sections 6.1–6.4, Oluwanifemi worked on Sections 6.5–6.6 and 7, Dayra worked on Section 9, and Harshith worked on Section 8. The team kept the same way of making decisions from P1.

We use messages for team communication and schedule online meetings when needed.

10.2 Plan to P3

Milestone	Date	Owner	Done when
Review P2 feedback	11/21/2026	All team members	Feedback has been reviewed
Review P3 instructions and requirements	11/25/2026	All team members	Everyone understands the P3 requirements
Divide work amongst team members	11/28/2026	All team members	Each member has an assigned task
Complete assigned tasks and check our progress	12/08/2026	All team members	Assigned tasks are completed and reviewed
Review the completed work, fix mistakes, and prepare the final submission	12/09/2026	All team members	Final work is checked and ready to submit

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1: Scope creep in P2	Closed	Low/High	Scope is firmly baselined in P2 and tracked via a requirements matrix Owner: Harshith
R2 - Unavailable university advisor for interviews	Open- no advisor interview was conducted	Medium/Low	Continue using the current requirements and record assumptions until feedback is available Owner: Samuel
R3 - Scheduling conflicts	Closed	Low/Medium	Communicating and planning tasks early and checking everyone's progress Owner: Ryan
R4 - Unable to finish planned features	Closed for P2	Low/Medium	Focus on important tasks and check progress before P3. Owner: Dayra
R5 - Problems combining work on GitHub	Closed: the team uploaded and combined their P2 without any conflict	Low/Low	Reviewing files before upload Owner: Oluwanifemi

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R6(new) - Dashboard performance	New risk tested by spike S-01	Medium/Medium	Test whether the dashboard responds within 2 seconds(QAS-01, S-01, ADR-01) Owner: Harshith
R7(new) - Document access security	New risk from separate document storage(ADR-02)	Medium/High	Check that student can only access their own uploaded documents(QAS-02, ADR-02) Owner: Krish

10.4 Repository

Our Github repository is https://github.com/nifemiolayiwola/InternTrack. We will keep our P2 document in /docs/p2, diagram files in /docs/p2/diagrams, and prototype work in /spikes/S-01. Each team member will work on their own branch and create a pull request for review before merging their changes. We will create a p2 tag that points to the commit containing the final PDF.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Krish	Claude code	Take my diagrams, and my section slides and push them to the appropriate github directories	Checked on github that section 5 PDF was uploaded to docs/p2 and the diagram files were uploaded to docs/p2/diagrams. Made sure to open the PDF and diagrams to confirm they were displayed correctly.
Oluwanifemi	ChatGPT	To organize the Github repository	I used CHATGPT to help me understand how to organize the github repository, create folders and upload files for P2. I followed the instructions and made changes myself.

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	Can students change an application status backward if they make a mistake?	All team members	Team, 11/08/2026	Closed: Students can change an application to any of the four status. Status changes are saved in the history	Students can correct a status mistake
Q2	Should passwords need more than 8 characters and include special characters?	All team members	Team, 11/08/2026	Open: The team has not decided on extra password rules.	Passwords must be at least 8 characters
Q3	Should advisors or recruiters be able to view student application progress?	All team members	Team, 11/08/2026	Closed: only students can access their own application progress	Advisors and recruiters do not have accounts or access.

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
Ryan Tracy	RT	11/08/2026
Samuel Rodriguez	SR	11/08/2026
Oluwanifemi Olayiwola	OO	11/08/2026
Dayra Ponce	DP	11/08/2026
Krish Joshi	KJ	11/08/2026
Harshith Reddy Tokala	HRT	11/08/2026